

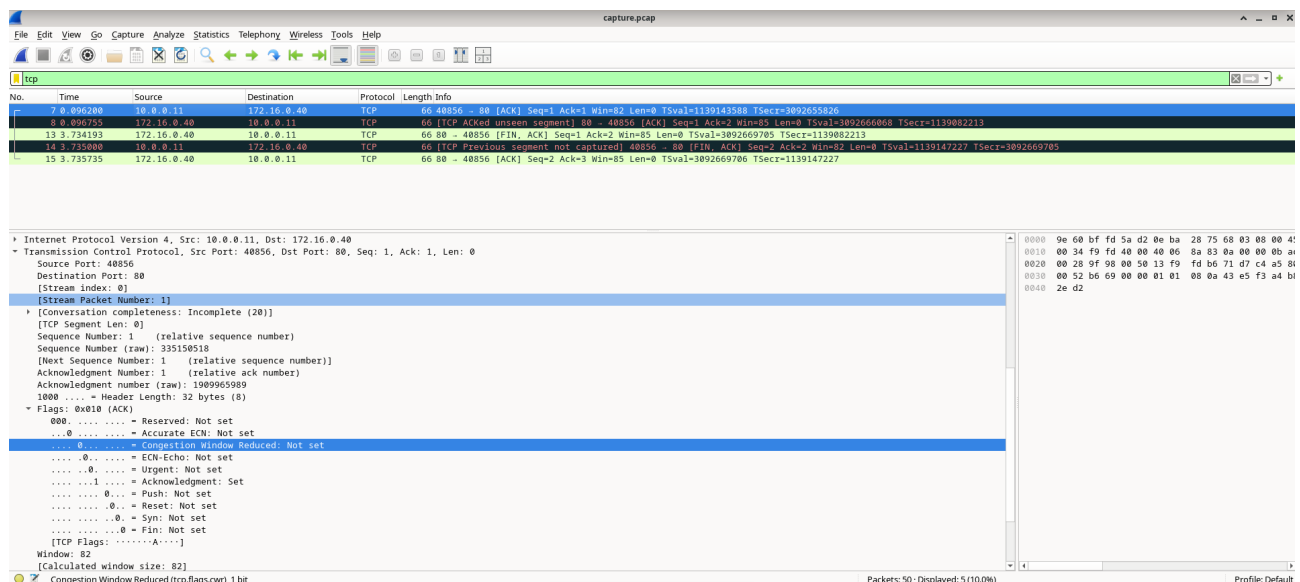
S11L2-Consegna

Analisi pacchetti Wireshark

1. Richiesta di chiusura del client (Pacchetto 13)

- Porta di Origine: 40856
- Porta di Destinazione: 80
- Flag Impostati:
 - FIN (Finish): Il client ha terminato di inviare dati.
 - ACK (Acknowledge): Conferma la ricezione dei dati precedentemente inviati dal server.
- Numero di Sequenza Relativo (Seq): 1
- Numero di Riconoscimento Relativo (Ack): 1

Descrizione: Questo è il primo passo del processo di chiusura. Il client (IP 10.0.0.11) invia questo pacchetto al server (IP 172.16.0.40) per segnalare che ha completato la sua parte di comunicazione e desidera terminare la connessione.



2. Riconoscimento del server (Pacchetto 14)

- Porta di Origine: 80
- Porta di Destinazione: 40856
- Flag Impostati:
 - ACK (Acknowledge): Il server conferma la ricezione del pacchetto FIN del client.
- Numero di Sequenza Relativo (Seq): 1
- Numero di Riconoscimento Relativo (Ack): 2

Descrizione: Il server (IP 172.16.0.40) risponde al client (IP 10.0.0.11) con un semplice ACK. A questo punto, la connessione è "a metà chiusa": il client non invierà più dati, ma il server potrebbe farlo se necessario.

The image shows a Wireshark packet capture window titled 'capture.pcap'. The packet list on the left shows five packets. Packet 14 is selected, showing a TCP segment from 172.16.0.40 to 10.0.0.11. The packet details pane on the right shows the following information:

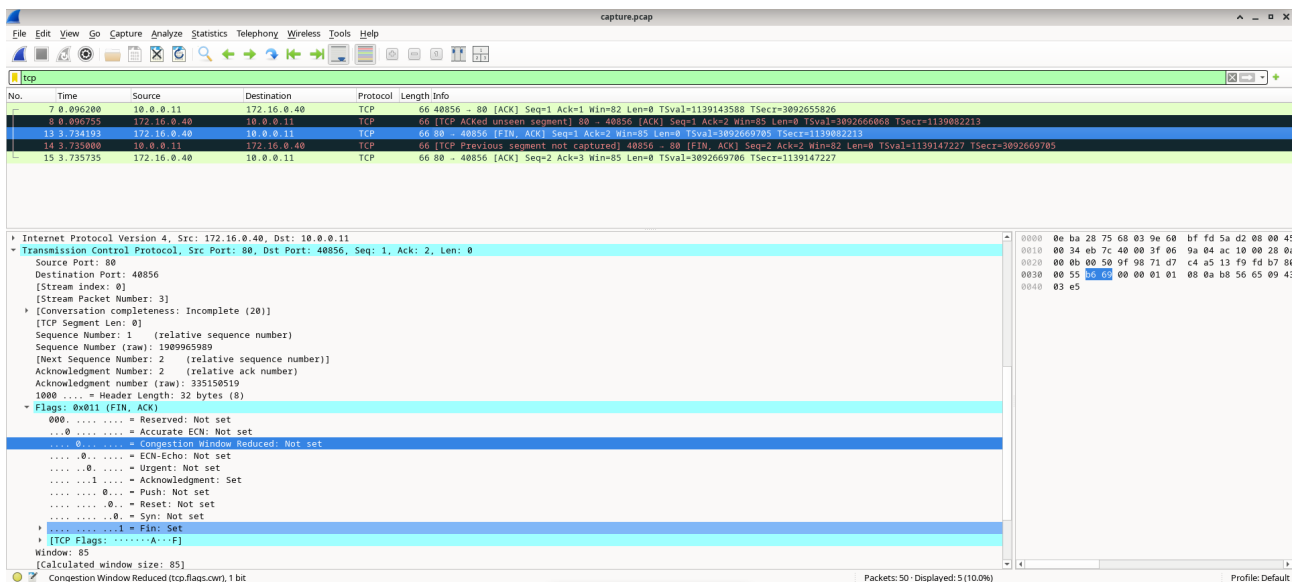
- Internet Protocol Version 4, Src: 172.16.0.40, Dst: 10.0.0.11
- Transmission Control Protocol, Src Port: 80, Dst Port: 40856, Seq: 1, Ack: 2, Len: 0
- Source Port: 80
- Destination Port: 40856
- [Stream Index: 0]
- [Stream Packet Number: 2]
- [Conversation completeness: Incomplete (20)]
- [TCP Segment Len: 0]
- Sequence Number: 1 (relative sequence number)
- Sequence Number (raw): 1909965989
- [Next Sequence Number: 1 (relative sequence number)]
- Acknowledgment Number: 2 (relative ack number)
- Acknowledgment number (raw): 335158519
- 1000 = Header Length: 32 bytes (8)
- Flags: 0x010 (ACK)
- 0000 = Reserved: Not set
-0.... = Accurate ECN: Not set
-0.... = Congestion Window Reduced: Not set
-0.... = ECN-Echo: Not set
-0.... = Urgent: Not set
-1.... = Acknowledgment: Set
-0.... = Push: Not set
-0.... = Reset: Not set
-0.... = Syn: Not set
-0.... = Fin: Not set
- [TCP Flags:A....]
- Window: 85
- [Calculated window size: 85]
- Congestion Window Reduced (tcp.flags.cwr), 1 bit

The packet bytes pane on the right shows the raw data of the packet, which is 32 bytes long (0x010).

3. Chiusura da parte del server (Pacchetto 15)

- Porta di Origine: 80
- Porta di Destinazione: 40856
- Flag Impostati:
 - FIN (Finish): Anche il server ha terminato l'invio dei dati e desidera chiudere la sua metà della connessione.
 - ACK (Acknowledge): Conferma la ricezione dei dati.
- Numero di Sequenza Relativo (Seq): 2
- Numero di Riconoscimento Relativo (Ack): 2

Descrizione: Il server (IP 172.16.0.40) invia questo pacchetto al client (IP 10.0.0.11) per completare la sua parte di chiusura. Con questo passaggio, la connessione è terminata da entrambe le direzioni. Un ulteriore pacchetto di ACK da parte del client confermerà al server l'avvenuta chiusura completa.



Analisi pacchetti tramite il comando tcpdump

```
TCPDUMP(1)                                General Commands Manual                                TCPDUMP(1)

NAME
    tcpdump - dump traffic on a network

SYNOPSIS
    tcpdump [ -AbDefhHIJKlLnNOpqStuUvxx# ] [ -B buffer size ]
    [ -c count ] [ --count ] [ -C file size ]
    [ -E spi@ipaddr algo:secret,... ]
    [ -F file ] [ -G rotate seconds ] [ -i interface ]
    [ --immediate-mode ] [ -j tstamp type ] [ -m module ]
    [ -M secret ] [ --number ] [ --print ] [ -Q in|out|inout ]
    [ -r file ] [ -s snaplen ] [ -T type ] [ --version ]
    [ -V file ] [ -w file ] [ -W filecount ] [ -y datalinktype ]
    [ -z postrotate-command ] [ -Z user ]
    [ --time-stamp-precision=tstamp precision ]
    [ --micro ] [ --nano ]
    [ expression ]
```

L'opzione -r viene utilizzata per leggere i pacchetti da un file salvato (in formato .pcap) invece di catturarli in tempo reale.

-r file

Read packets from file (which was created with the -w option or by other tools that write pcap or pcapng files). Standard input is used if file is '-'.

L'output di tcpdump mostra un handshake TCP a tre vie in corso:

- Pacchetto 1 (SYN): 10.0.0.11 > 172.16.0.40.http: Flags [S]... Il client 10.0.0.11 avvia la connessione inviando un pacchetto SYN (Synchronize) al server 172.16.0.40 sulla porta 80 (HTTP).
- Pacchetto 2 (SYN, ACK): 172.16.0.40.http > 10.0.0.11: Flags [S.], ack 1... Il server risponde con un pacchetto SYN, ACK, riconoscendo la richiesta del client e inviando la propria richiesta di sincronizzazione.
- Pacchetto 3 (ACK): 10.0.0.11 > 172.16.0.40.http: Flags [.], ack 1... Il client completa l'handshake inviando un pacchetto ACK finale, che stabilisce la connessione.

```

File Edit View Terminal Tabs Help
[analyst@secOps ~]$ tcpdump -r /home/analyst/capture.pcap tcp -c 3
reading from file /home/analyst/capture.pcap, link-type EN10MB (Ethernet), snapshot length 262144
12:20:29.196190 IP 10.0.0.11.40856 > 172.16.0.40.http: Flags [.], ack 1909965989, win 82, options [nop,nop,TS val 1139143588 ecr 3092655826], length 0
12:20:29.196745 IP 172.16.0.40.http > 10.0.0.11.40856: Flags [.], ack 1, win 85, options [nop,nop,TS val 3092666068 ecr 1139082213], length 0
12:20:32.834183 IP 172.16.0.40.http > 10.0.0.11.40856: Flags [F.], seq 1, ack 1, win 85, options [nop,nop,TS val 3092669705 ecr 1139082213], length 0
[analyst@secOps ~]$ sudo mn -c
[sudo] password for analyst:
*** Removing excess controllers/ofprotocols/ofdatapaths/pings/noxes
killall controller ofprotocol ofdatapath ping nox_core lt-nox_core ovs-openflowd ovs-controller ovs-testcontroller udpbwtest mnexec ivs ryu-manager 2> /dev/null
killall -9 controller ofprotocol ofdatapath ping nox_core lt-nox_core ovs-openflowd ovs-controller ovs-testcontroller udpbwtest mnexec ivs ryu-manager 2> /dev/null
pkill -9 -f "sudo mnexec"
*** Removing junk from /tmp
rm -f /tmp/vconn* /tmp/vlogs* /tmp/*.out /tmp/*.log
*** Removing old X11 tunnels
*** Removing excess kernel datapaths
ps ax | egrep -o 'dp[0-9]+' | sed 's/dp/nl:/'
egrep: warning: egrep is obsolescent; using grep -E
*** Removing OVS datapaths
ovs-vsctl --timeout=1 list-br
ovs-vsctl --if-exists del-br s1
ovs-vsctl --timeout=1 list-br
*** Removing all links of the pattern foo-ethX
ip link show | egrep -o '([0-9a-f]{4}+eth[[:digit:]]+)'
egrep: warning: egrep is obsolescent; using grep -E
( ip link del s1-eth2; ip link del s1-eth3; ip link del s1-eth4; ip link del s1-eth1 ) 2> /dev/null
ip link show
*** Killing stale mininet node processes
pkill -9 -f mininet:
*** Shutting down stale tunnels
pkill -9 -f Tunnel=Ethernet
pkill -9 -f .ssh/mn/
rm -f ~/.ssh/mn/*
*** Cleanup complete.
[analyst@secOps ~]$

```

Tre filtri utili per un amministratore di rete:

- `ip.add ==` indirizzo ip specifico: Isola tutto il traffico da e verso uno specifico host. È essenziale per la risoluzione dei problemi su un singolo server o per indagare su attività sospette.
- `tcp.flags.syn == 1 and tcp.flags.ack == 0`: Mostra solo i tentativi di avvio della connessione. Utile per rilevare scansioni delle porte o attacchi di tipo SYN flood.
- `http or dns`: Mostra il traffico web non crittografato e le query DNS. Aiuta a comprendere l'uso della rete e a identificare potenziali problemi di sicurezza.

Oltre alla risoluzione di problemi di base, Wireshark viene utilizzato in una rete di produzione per:

- Analisi delle prestazioni: Identificare la latenza di rete, la perdita di pacchetti o i colli di bottiglia nelle prestazioni delle applicazioni.
- Audit di sicurezza: Rilevare protocolli non sicuri (come HTTP o FTP), traffico non crittografato e identificare comunicazioni con indirizzi IP malevoli conosciuti.
- Analisi dei protocolli: Comprendere come comunicano le applicazioni specifiche e assicurarsi che siano conformi alle policy di rete.